ログイン画面UI設計書

メタデータ

title: ログイン画面UI設計書
category: UI設計
version: 1.0.0
last_updated: 2025-12-29
author: Development Team
tags: [authentication, login, 2fa, ui, client-certificate]

関連ドキュメント

- 認証API仕様
- 認証DB設計
- セッション管理比較
- 技術仕様書
- セキュリティ規約
- UIポリシー

---

1. 概要

1.1. 目的

mmdpfプロジェクトのログイン認証画面を定義します。クライアント証明書認証、パスワード認証、二段階認証（メールPINコード）を組み合わせた安全な認証システムを提供します。

1.2. 認証方式の特徴

クライアント証明書認証（必須）

- nginx層でTLS/SSL証明書を検証
- 証明書のCN（Common Name）をユーザー識別に使用
- 証明書のOU（Organization Unit）を組織ID として抽出
- ユーザーには組織選択を表示しない（透過的）

二段階認証（2FA）

- メールでPINコード（6桁数字）を送信
- PINコードの有効期限: 5分
- 最大試行回数: 3回

メールアドレス未登録時の運用

- ユーザー本人のメールアドレスが未登録の場合、管理者のメールアドレスに送信
- 管理者が口頭でPINコードをユーザーに伝達

1.3. 認証フロー概要

```
[1. ログイン画面]
    ↓ ユーザー名・パスワード入力
[nginx] クライアント証明書検証
    ↓ 証明書OK
[2. パスワード認証]
    ↓ パスワードOK
[3. PINコード生成・メール送信]
    ↓
[4. 二段階認証画面]
    ↓ PINコード入力
[5. セッション生成]
    ↓
[6. ロール別画面へ遷移]
```

---

2. 画面構成

2.1. 画面一覧

画面名	URL	説明
ログイン画面	`/login`	ユーザー名・パスワード入力
二段階認証画面	`/login/verify`	PINコード入力
新規ユーザー登録画面	`/register`	新規ユーザー情報登録（将来実装）

---

3. ログイン画面（Step 1）

3.1. 画面概要

URL: `/login`

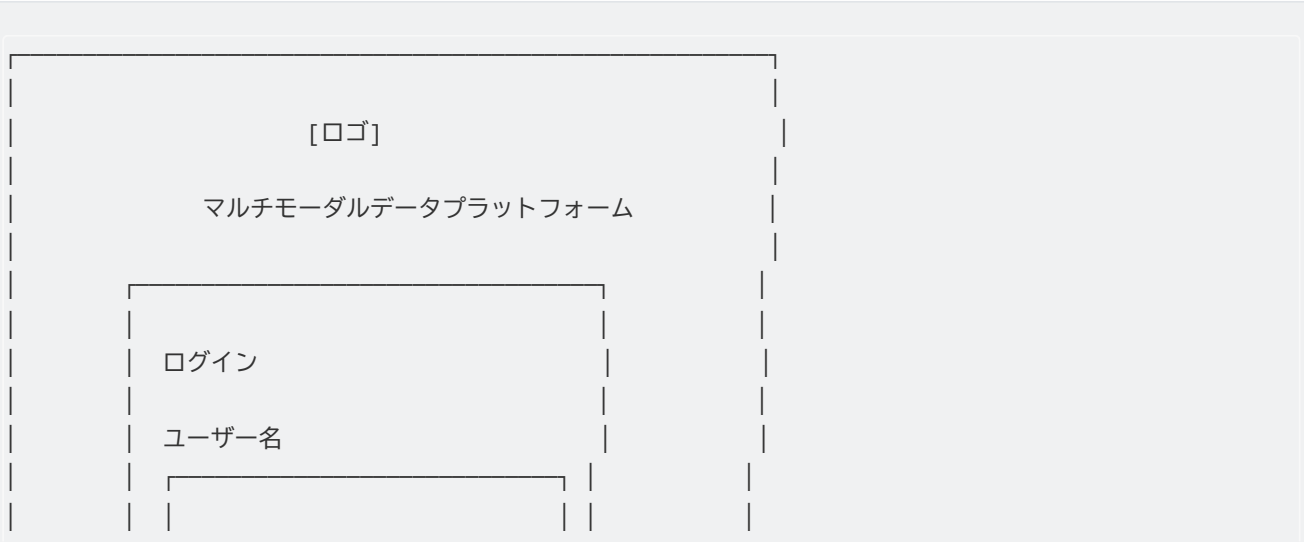
目的: ユーザー名とパスワードを入力し、初回認証を行う

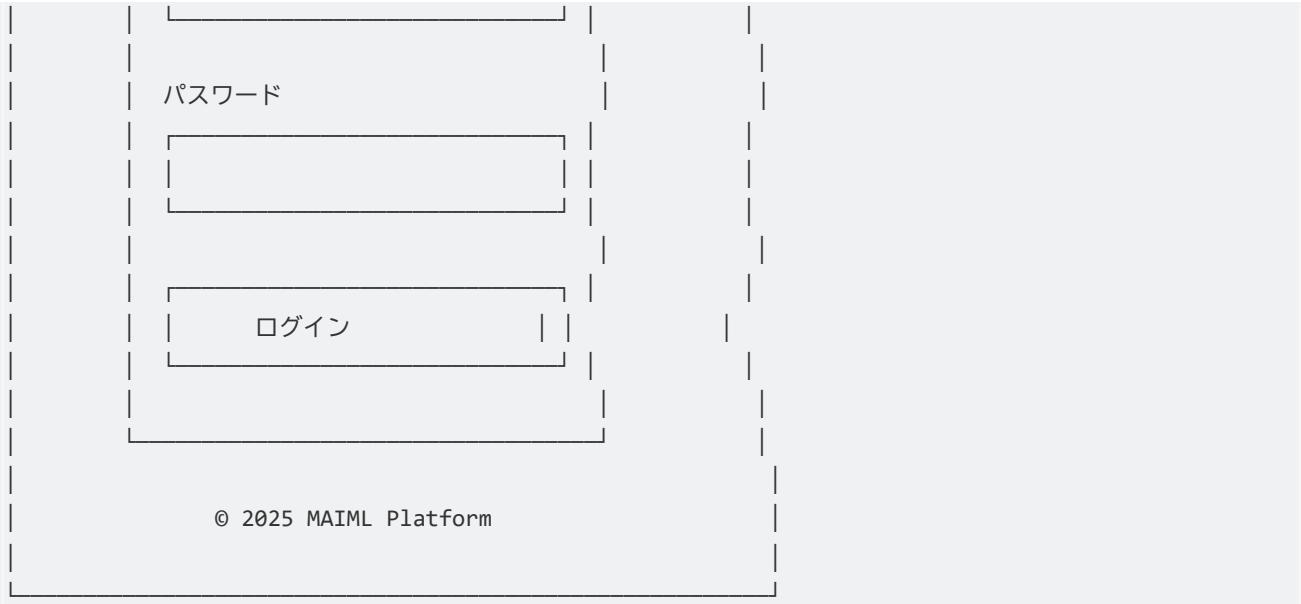
前提条件:

- クライアント証明書がインストールされている
- nginxが証明書を検証済み

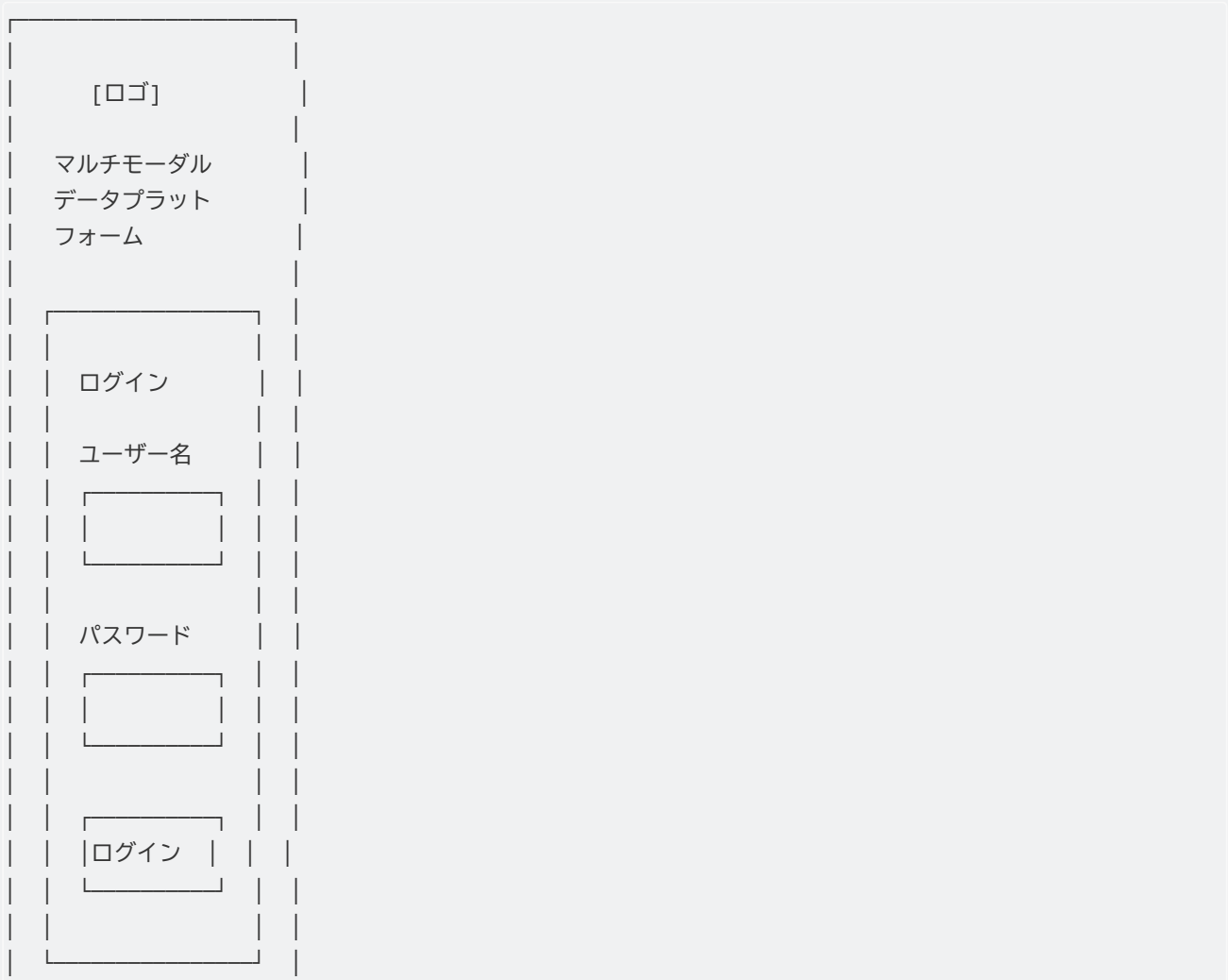
3.2. レイアウト

デスクトップ（lg, xl, xxl）




パスワード
ログイン
© 2025 MAIML Platform

モバイル（xs, sm）


[ロゴ]
マルチモーダル
データプラット
フォーム
ログイン
ユーザー名
パスワード
ログイン

```
│                          │
└──────────────────────────┘
```

3.3. コンポーネント構成

ロゴエリア

- **コンポーネント**: `<div>` + `<Image>`
- **サイズ**: 200px × 80px（推奨）
- **配置**: 中央揃え

タイトル

- **コンポーネント**: `<Typography.Title level={2}>`
- **テキスト**:「マルチモーダルデータプラットフォーム」
- **配置**: 中央揃え

ログインカード

- **コンポーネント**: `<Card>`
- **幅**:
  - デスクトップ: 400px
  - モバイル: 100%（最大400px）
- **配置**: 中央揃え
- **スタイル**: `bordered={false}`, `hoverable={false}`

3.4. 入力項目

3.4.1. ユーザー名（必須）

コンポーネント: `<Form.Item>` + `<Input>`

仕様:

項目	値
ラベル	「ユーザー名」
プレースホルダー	「ユーザー名を入力」
型	text
必須	○
最小文字数	1文字
最大文字数	100文字
許可文字	英数字、アンダースコア、ハイフン
前置アイコン	`<UserOutlined />`

バリデーション:

```
{
  required: true,
  message: 'ユーザー名を入力してください'
},
{
  min: 1,
  max: 100,
  message: 'ユーザー名は1〜100文字で入力してください'
},
{
  pattern: /^[a-zA-Z0-9_-]+$/,
  message: 'ユーザー名は英数字、アンダースコア、ハイフンのみ使用できます'
}
```

3.4.2. パスワード（必須）

コンポーネント: `<Form.Item>` + `<Input.Password>`

仕様:

項目	値
ラベル	「パスワード」
プレースホルダー	「パスワードを入力」
型	password
必須	○

最小文字数	8文字
最大文字数	255文字
前置アイコン	`<LockOutlined />`
可視化トグル	有効

バリデーション:

```
{
  required: true,
  message: 'パスワードを入力してください'
},
{
  min: 8,
  max: 255,
  message: 'パスワードは8〜255文字で入力してください'
}
```

3.5. アクションボタン

ログインボタン

コンポーネント: `<Button>`

仕様:

項目	値
ラベル	「ログイン」
タイプ	`primary`
サイズ	`large`
幅	100%
htmlType	`submit`
ローディング	送信中は `true`
アイコン	`<LoginOutlined />`

動作:

1. フォームバリデーション実行
2. ローディング状態に変更
3. API呼び出し: `POST /api/v1/auth/login`
4. 成功時: 二段階認証画面へ遷移
5. 失敗時: エラーメッセージ表示

3.6. エラーメッセージ

エラー表示エリア

コンポーネント: `<Alert>`

仕様:

項目	値
type	`error`
showIcon	`true`
closable	`true`
message	エラーメッセージ

表示条件:

- API呼び出し失敗時
- バリデーションエラー時

エラーメッセージ一覧:

エラーコード	メッセージ	説明
`AUTH_INVALID_CREDENTIALS`	「ユーザー名またはパスワードが正しくありません」	認証失敗
`AUTH_CERTIFICATE_INVALID`	「クライアント証明書が無効です」	証明書エラー
`AUTH_CERTIFICATE_EXPIRED`	「クライアント証明書の有効期限が切れています」	証明書期限切れ
`AUTH_ACCOUNT_DISABLED`	「このアカウントは無効化されています」	アカウント無効

`AUTH_RATE_LIMIT_EXCEEDED`	「ログイン試行回数が上限に達しました。15分後に再試行してください」	レート制限
`SYSTEM_EMAIL_SEND_FAILED`	「メール送信に失敗しました。しばらく経ってから再試行してください」	メール送信失敗
`VALIDATION_ERROR`	「入力内容に誤りがあります」	バリデーションエラー

3.7. 証明書エラー時の特別な表示

クライアント証明書が検証できない場合（nginxレベルでブロック）：

コンポーネント: `<Result>`

```
<Result
  status="error"
  title="クライアント証明書が検証できません"
  subTitle="有効なクライアント証明書がインストールされていません。システム管理者にお問い合わせください。"
  extra={[
    <Button type="primary" key="contact">
      サポートに連絡
    </Button>
  ]}
/>
```

3.8. APIリクエスト仕様

エンドポイント

`POST /api/v1/auth/login`

リクエストヘッダー

```
Content-Type: application/json
X-Organization-Id: org_maiml_lab_001  # nginx経由で自動付与
```

リクエストボディ

```
{
  "username": "user001",
  "password": "SecurePassword123!"
}
```

レスポンス（成功時）

```
{
  "success": true,
  "data": {
    "sessionToken": "temp_session_abc123_1735185000",
    "message": "PINコードをメールに送信しました",
    "email": "use***@example.com",
    "expiresIn": 300
  },
  "meta": {
    "timestamp": "2025-12-29T10:30:00.000Z",
    "requestId": "req_abc123"
  }
}
```

レスポンス（エラー時）

```
{
  "success": false,
  "error": {
    "code": "AUTH_INVALID_CREDENTIALS",
    "message": "ユーザー名またはパスワードが正しくありません",
    "details": []
  },
  "meta": {
    "timestamp": "2025-12-29T10:30:00.000Z",
    "requestId": "req_abc123"
  }
}
```

---

4. 二段階認証画面（Step 2）

4.1. 画面概要

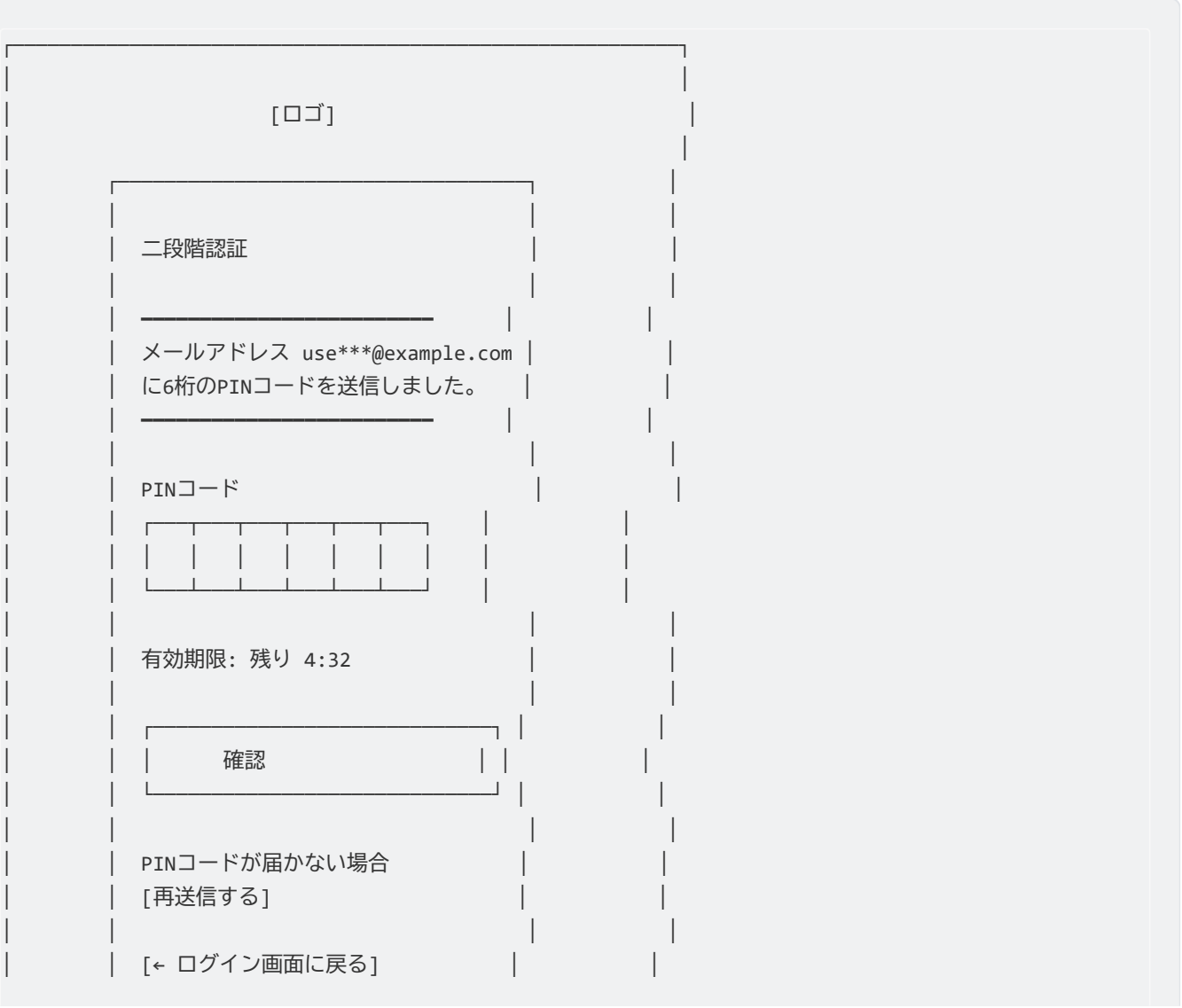
URL: `/login/verify`

目的: メールで受信したPINコード（6桁数字）を入力し、二段階認証を完了する

前提条件:

- ログイン画面でユーザー名・パスワード認証が成功している
- 仮セッショントークンを保持している

4.2. レイアウト



```
│        │                                  │           │
│        └──────────────────────────────────┘           │
│                                                       │
└───────────────────────────────────────────────────────┘
```

4.3. コンポーネント構成

情報表示エリア

コンポーネント: `<Alert>`

仕様:

項目	値
type	`info`
showIcon	`true`
message	「PINコードを送信しました」
description	「メールアドレス use***@example.com に6桁のPINコードを送信しました。メールをご確認ください。」

メールアドレス未登録時の特別メッセージ:

```
「メールアドレスが登録されていないため、システム管理者にPINコードを送信しました。
管理者からPINコードを受け取ってください。」
```

4.4. 入力項目

4.4.1. PINコード（必須）

コンポーネント: `<Input.OTP>` (Ant Design v5.21+)

仕様:

項目	値
ラベル	「PINコード」
桁数	6桁
型	number

必須	○
自動フォーカス	有効
マスク表示	無効（数字のまま表示）

バリデーション:

```
{
  required: true,
  message: 'PINコードを入力してください'
},
{
  pattern: /^[0-9]{6}$/,
  message: 'PINコードは6桁の数字です'
}
```

自動送信:

- 6桁入力完了時に自動的に検証APIを呼び出す

4.4.2. 有効期限タイマー

コンポーネント: `<Statistic.Countdown>`

仕様:

項目	値
フォーマット	「mm:ss」
初期値	5:00（5分）
完了時動作	エラーメッセージ表示 + 再送信ボタン有効化

スタイル:

- フォントサイズ: 16px
- 色:
  - 残り3分以上: `colorTextSecondary`
  - 残り1〜3分: `colorWarning`

- 残り1分未満: `colorError`

4.5. アクションボタン

確認ボタン

コンポーネント: `<Button>`

仕様:

項目	値
ラベル	「確認」
タイプ	`primary`
サイズ	`large`
幅	100%
htmlType	`submit`
ローディング	検証中は `true`
無効化	PINコード6桁未入力時

再送信ボタン

コンポーネント: `<Button>`

仕様:

項目	値
ラベル	「再送信する」
タイプ	`link`
サイズ	`middle`
ローディング	再送信中は `true`
クールダウン	再送信後60秒間無効化

クールダウン表示:

「再送信する（60秒後に再試行可能）」

戻るボタン

コンポーネント: `<Button>`

仕様:

項目	値
ラベル	「← ログイン画面に戻る」
タイプ	`text`
サイズ	`middle`
アイコン	`<ArrowLeftOutlined />`

動作:

- `/login` へ遷移
- 仮セッショントークンを破棄

4.6. エラーメッセージ

エラーメッセージ一覧:

エラーコード	メッセージ	説明
`AUTH_PIN_INVALID`	「PINコードが正しくありません」	PIN不一致
`AUTH_PIN_EXPIRED`	「PINコードの有効期限が切れました。再送信してください」	有効期限切れ
`AUTH_PIN_MAX_ATTEMPTS`	「PINコードの入力試行回数が上限に達しました。最初からやり直してください」	試行回数超過
`AUTH_TOKEN_INVALID`	「セッションが無効です。最初からやり直してください」	トークンエラー

4.7. APIリクエスト仕様

エンドポイント

`POST /api/v1/auth/verify-pin`

リクエストヘッダー

```
Content-Type: application/json
Authorization: Bearer temp_session_abc123_1735185000
X-Organization-Id: org_maiml_lab_001
```

リクエストボディ

```
{
  "pinCode": "123456"
}
```

レスポンス（成功時）

```
{
  "success": true,
  "data": {
    "user": {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "username": "user001",
      "email": "user001@example.com",
      "role": "experiment_admin",
      "organizationId": "org_maiml_lab_001",
      "organizationName": "MAIML研究室",
      "displayName": "山田 太郎"
    },
    "session": {
      "token": "maiml_session-abc123_1735185000",
      "expiresAt": "2025-12-30T10:30:00.000Z"
    },
    "redirectUrl": "/experiment-admin"
  },
  "meta": {
    "timestamp": "2025-12-29T10:30:00.000Z",
    "requestId": "req_abc123"
```

```
  }
}
```

Set-Cookie ヘッダー:

```
Set-Cookie: maiml_session=maiml_session-abc123_1735185000; HttpOnly; Secure; SameSite=Strict;
Max-Age=86400; Path=/
```

---

5. 新規ユーザー登録画面（将来実装）

5.1. 画面概要

URL: `/register`

目的: 新規ユーザーの情報を登録する

実装状況: 将来実装予定

想定機能:

- ユーザー名入力
- メールアドレス入力
- パスワード設定
- パスワード確認
- 利用規約同意

---

6. 画面遷移フロー

6.1. 正常フロー

```
[ログイン画面]
    ↓ ユーザー名・パスワード入力 → ログインボタンクリック
[nginx] 証明書検証
```

```
    ↓ 証明書OK
[API] POST /api/v1/auth/login
    ↓ パスワードOK、PINコード送信
[二段階認証画面]
    ↓ PINコード入力 → 確認ボタンクリック（または自動送信）
[API] POST /api/v1/auth/verify-pin
    ↓ PIN OK
[ロール別画面]
    ├ provider → /provider
    ├ system_admin → /system-admin
    ├ data_admin → /data-admin
    ├ experiment_admin → /experiment-admin
    ├ experiment_worker → /experiment-worker
    └ mi_engineer → /mi-engineer
```

6.2. エラーフロー

証明書エラー

```
[ログイン画面]
    ↓ ページ読み込み
[nginx] 証明書検証
    ↓ 証明書NG
[証明書エラー画面]
    表示: "クライアント証明書が検証できません"
```

パスワードエラー

```
[ログイン画面]
    ↓ ユーザー名・パスワード入力 → ログインボタンクリック
[API] POST /api/v1/auth/login
    ↓ パスワードNG
[ログイン画面]
    エラーメッセージ表示: "ユーザー名またはパスワードが正しくありません"
```

PINコードエラー

```
[二段階認証画面]
    ↓ PINコード入力 → 確認ボタンクリック
[API] POST /api/v1/auth/verify-pin
    ↓ PIN NG
```

```
[二段階認証画面]
    エラーメッセージ表示: "PINコードが正しくありません"
    試行回数: 残り2回 → 残り1回 → 残り0回
    ↓ 試行回数超過
[二段階認証画面]
    エラーメッセージ表示: "入力試行回数が上限に達しました"
    ↓ 戻るボタンクリック
[ログイン画面]
```

PIN有効期限切れ

```
[二段階認証画面]
    ↓ 5分経過
[タイマー] 00:00になる
    ↓
[二段階認証画面]
    エラーメッセージ表示: "PINコードの有効期限が切れました"
    再送信ボタン有効化
    ↓ 再送信ボタンクリック
[API] POST /api/v1/auth/resend-pin
    ↓ 新しいPINコード送信
[二段階認証画面]
    タイマーリセット: 5:00
```

---

7. レスポンシブデザイン

7.1. ブレークポイント

サイズ	幅	カード幅	フォントサイズ
xs	< 576px	100%	14px
sm	576px ~ 768px	100%	14px
md	768px ~ 992px	400px	14px
lg	992px ~ 1200px	400px	14px
xl	1200px ~ 1600px	400px	16px
xxl	> 1600px	400px	16px

7.2. モバイル対応

タッチターゲット

- 最小サイズ: 44px × 44px（Apple HIG準拠）
- ボタン間の間隔: 16px以上

キーボード表示

- PINコード入力時: 数字キーボードを自動表示
- パスワード入力時: 標準キーボード

スクロール対応

- カードが画面内に収まらない場合は垂直スクロール
- 入力フォーカス時にキーボードで隠れないように自動スクロール

---

8. アクセシビリティ

8.1. キーボード操作

Tab順序

1. ユーザー名入力
2. パスワード入力
3. ログインボタン
4. その他のリンク・ボタン

ショートカットキー

- Enter: フォーム送信
- Escape: エラーメッセージを閉じる

8.2. スクリーンリーダー対応

ARIA属性

```
<Form.Item
  label="ユーザー名"
  name="username"
  rules={[...]}
>
  <Input
    prefix={<UserOutlined />}
    placeholder="ユーザー名を入力"
    aria-label="ユーザー名"
    aria-required="true"
    aria-describedby="username-error"
  />
</Form.Item>
```

エラーメッセージの読み上げ

- `role="alert"` を使用
- エラー発生時に自動的にフォーカス

8.3. コントラスト比

要素	背景色	前景色	コントラスト比
通常テキスト	`#FFFFFF`	`#000000`	21:1 ✅
ボタン（Primary）	`#CF0027`	`#FFFFFF`	4.8:1 ✅
リンク	`#FFFFFF`	`#1677FF`	4.5:1 ✅
エラーテキスト	`#FFFFFF`	`#FF4D4F`	4.5:1 ✅

WCAG 2.1 AA基準（4.5:1以上）を満たす

---

9. セキュリティ考慮事項

9.1. クライアントサイド

パスワード入力

- ブラウザの自動保存機能に対応（`autocomplete="current-password"`）
- コピー&ペーストを許可（UX向上のため）
- 入力内容はマスク表示（`<Input.Password>`）

セッショントークン

- LocalStorageには保存しない（セキュリティリスク）
- HTTPOnly Cookieで管理
- XSS攻撃対策

入力値のサニタイゼーション

- HTMLタグは自動エスケープ（React標準機能）
- SQLインジェクション対策はバックエンドで実施

9.2. レート制限の表示

ログイン試行回数が上限に達した場合の表示:

```
<Alert
  type="error"
  showIcon
  message="ログイン試行回数が上限に達しました"
  description={
    <>
      <p>セキュリティのため、一時的にログインを制限しています。</p>
      <p>15分後に再度お試しください。</p>
      <p>次回リトライ可能時刻: {retryAfterTime}</p>
    </>
  }
/>
```

9.3. HTTPS必須

本番環境では必ずHTTPS通信を使用:

- クライアント証明書の検証
- Cookie の Secure フラグ
- パスワードの安全な送信

---

10. パフォーマンス最適化

10.1. ローディング状態

初回ローディング

- ログインカードのみを先に表示
- ロゴ画像は遅延読み込み（lazy loading）

API呼び出し中

```
<Button
  type="primary"
  size="large"
  htmlType="submit"
  loading={isLoading}
  disabled={isLoading}
  block
>
  {isLoading ? 'ログイン中...' : 'ログイン'}
</Button>
```

10.2. エラーリトライ

ネットワークエラー時の自動リトライ:

- 最大3回まで自動リトライ

- Exponential Backoff（指数バックオフ）

10.3. プリフェッチ

二段階認証画面へ遷移する前に:

- PINコード入力画面のコンポーネントをプリロード
- 遷移をスムーズに

---

11. 多言語化（i18n）

11.1. 対応言語

初期バージョン:

- 日本語のみ

将来対応予定:

- 英語
- 中国語

11.2. 翻訳キー

```
// frontend/src/locales/ja.json
{
  "auth": {
    "login": {
      "title": "ログイン",
      "username": "ユーザー名",
      "password": "パスワード",
      "loginButton": "ログイン",
      "errors": {
        "invalidCredentials": "ユーザー名またはパスワードが正しくありません",
        "certificateInvalid": "クライアント証明書が無効です"
      }
    },
    "twoFactor": {
```

```
      "title": "二段階認証",
      "pinCode": "PINコード",
      "verifyButton": "確認",
      "resendButton": "再送信する",
      "backButton": "ログイン画面に戻る"
    }
  }
}
```

---

12. 実装例（TypeScript + Next.js + Ant Design）

12.1. ログイン画面コンポーネント

```tsx
// frontend/src/app/login/page.tsx
'use client';

import React, { useState } from 'react';
import { Form, Input, Button, Card, Typography, Alert, Space } from 'antd';
import { UserOutlined, LockOutlined, LoginOutlined } from '@ant-design/icons';
import { useRouter } from 'next/navigation';
import apiClient from '@/utils/apiClient';

const { Title, Text } = Typography;

interface LoginFormValues {
  username: string;
  password: string;
}

export default function LoginPage() {
  const router = useRouter();
  const [form] = Form.useForm();
  const [isLoading, setIsLoading] = useState(false);
  const [error, setError] = useState<string | null>(null);

  const handleLogin = async (values: LoginFormValues) => {
    setIsLoading(true);
    setError(null);

    try {
      const response = await apiClient.post('/api/v1/auth/login', {
        username: values.username,
        password: values.password
```

```
      });

      if (response.data.success) {
        // 仮セッショントークンを保存
        sessionStorage.setItem('tempToken', response.data.data.sessionToken);
        sessionStorage.setItem('email', response.data.data.email);

        // 二段階認証画面へ遷移
        router.push('/login/verify');
      }
    } catch (err: any) {
      const errorCode = err.response?.data?.error?.code;
      const errorMessage = err.response?.data?.error?.message;

      setError(errorMessage || 'ログインに失敗しました');
    } finally {
      setIsLoading(false);
    }
  };

  return (
    <div style={{
      minHeight: '100vh',
      display: 'flex',
      flexDirection: 'column',
      alignItems: 'center',
      justifyContent: 'center',
      backgroundColor: '#f0f2f5'
    }}>
      {/* ロゴエリア */}
      <Space direction="vertical" size="large" align="center" style={{ width: '100%',
maxWidth: 400, padding: '0 16px' }}>
        {/* ロゴ画像（将来実装） */}
        <div style={{ width: 200, height: 80, backgroundColor: '#e0e0e0', borderRadius: 8 }}
/>

        <Title level={2} style={{ textAlign: 'center', margin: 0 }}>
          マルチモーダルデータプラットフォーム
        </Title>

        {/* ログインカード */}
        <Card
          bordered={false}
          style={{ width: '100%', maxWidth: 400 }}
        >
          <Title level={3}>ログイン</Title>

          {/* エラーメッセージ */}
          {error && (
            <Alert
```

```
              type="error"
              showIcon
              closable
              message={error}
              onClose={() => setError(null)}
              style={{ marginBottom: 16 }}
            />
          )}

          <Form
            form={form}
            layout="vertical"
            onFinish={handleLogin}
            autoComplete="off"
          >
            {/* ユーザー名 */}
            <Form.Item
              label="ユーザー名"
              name="username"
              rules={[
                { required: true, message: 'ユーザー名を入力してください' },
                { min: 1, max: 100, message: 'ユーザー名は1〜100文字で入力してください' },
                { pattern: /^[a-zA-Z0-9_-]+$/, message: 'ユーザー名は英数字、アンダースコア、ハイフンのみ使用できます' }
              ]}
            >
              <Input
                prefix={<UserOutlined />}
                placeholder="ユーザー名を入力"
                size="large"
                autoComplete="username"
              />
            </Form.Item>

            {/* パスワード */}
            <Form.Item
              label="パスワード"
              name="password"
              rules={[
                { required: true, message: 'パスワードを入力してください' },
                { min: 8, max: 255, message: 'パスワードは8〜255文字で入力してください' }
              ]}
            >
              <Input.Password
                prefix={<LockOutlined />}
                placeholder="パスワードを入力"
                size="large"
                autoComplete="current-password"
              />
            </Form.Item>
```

```
            {/* ログインボタン */}
            <Form.Item style={{ marginBottom: 0 }}>
              <Button
                type="primary"
                size="large"
                htmlType="submit"
                loading={isLoading}
                disabled={isLoading}
                icon={<LoginOutlined />}
                block
              >
                {isLoading ? 'ログイン中...' : 'ログイン'}
              </Button>
            </Form.Item>
          </Form>
        </Card>

        {/* フッター */}
        <Text type="secondary" style={{ textAlign: 'center' }}>
          © 2025 MAIML Platform
        </Text>
      </Space>
    </div>
  );
}
```

12.2. 二段階認証画面コンポーネント

```
// frontend/src/app/login/verify/page.tsx
'use client';

import React, { useState, useEffect } from 'react';
import { Form, Button, Card, Typography, Alert, Space, Statistic, Input } from 'antd';
import { ArrowLeftOutlined, CheckOutlined } from '@ant-design/icons';
import { useRouter } from 'next/navigation';
import apiClient from '@/utils/apiClient';

const { Title, Text } = Typography;
const { Countdown } = Statistic;

export default function VerifyPinPage() {
  const router = useRouter();
  const [form] = Form.useForm();
  const [isLoading, setIsLoading] = useState(false);
  const [isResending, setIsResending] = useState(false);
```

```
  const [error, setError] = useState<string | null>(null);
  const [email, setEmail] = useState<string>('');
  const [expiresAt, setExpiresAt] = useState<number>(Date.now() + 5 * 60 * 1000);
  const [resendCooldown, setResendCooldown] = useState<number>(0);

  useEffect(() => {
    // 仮セッショントークンとメールアドレスを取得
    const storedEmail = sessionStorage.getItem('email');
    const tempToken = sessionStorage.getItem('tempToken');

    if (!tempToken) {
      // トークンがない場合はログイン画面へ戻る
      router.push('/login');
      return;
    }

    if (storedEmail) {
      setEmail(storedEmail);
    }
  }, [router]);

  const handleVerifyPin = async (values: { pinCode: string }) => {
    setIsLoading(true);
    setError(null);

    try {
      const tempToken = sessionStorage.getItem('tempToken');

      const response = await apiClient.post(
        '/api/v1/auth/verify-pin',
        { pinCode: values.pinCode },
        {
          headers: {
            Authorization: `Bearer ${tempToken}`
          }
        }
      );

      if (response.data.success) {
        // セッショントークンはCookieに自動設定される
        sessionStorage.removeItem('tempToken');
        sessionStorage.removeItem('email');

        // ロール別画面へ遷移
        const redirectUrl = response.data.data.redirectUrl;
        router.push(redirectUrl);
      }
    } catch (err: any) {
      const errorMessage = err.response?.data?.error?.message;
      setError(errorMessage || 'PINコードの検証に失敗しました');
```

```
    // エラー時はフォームをクリア
    form.resetFields();
  } finally {
    setIsLoading(false);
  }
};

const handleResendPin = async () => {
  if (resendCooldown > 0) return;

  setIsResending(true);
  setError(null);

  try {
    const tempToken = sessionStorage.getItem('tempToken');

    const response = await apiClient.post(
      '/api/v1/auth/resend-pin',
      {},
      {
        headers: {
          Authorization: `Bearer ${tempToken}`
        }
      }
    );

    if (response.data.success) {
      // タイマーをリセット
      setExpiresAt(Date.now() + 5 * 60 * 1000);

      // 再送信クールダウンを開始
      setResendCooldown(60);
      const interval = setInterval(() => {
        setResendCooldown(prev => {
          if (prev <= 1) {
            clearInterval(interval);
            return 0;
          }
          return prev - 1;
        });
      }, 1000);
    }
  } catch (err: any) {
    const errorMessage = err.response?.data?.error?.message;
    setError(errorMessage || 'PINコードの再送信に失敗しました');
  } finally {
    setIsResending(false);
  }
};
```

```
  const handleBack = () => {
    sessionStorage.removeItem('tempToken');
    sessionStorage.removeItem('email');
    router.push('/login');
  };

  return (
    <div style={{
      minHeight: '100vh',
      display: 'flex',
      flexDirection: 'column',
      alignItems: 'center',
      justifyContent: 'center',
      backgroundColor: '#f0f2f5'
    }}>
      <Space direction="vertical" size="large" align="center" style={{ width: '100%',
maxWidth: 400, padding: '0 16px' }}>
        {/* ロゴ画像（将来実装） */}
        <div style={{ width: 200, height: 80, backgroundColor: '#e0e0e0', borderRadius: 8 }}
/>

        {/* 二段階認証カード */}
        <Card bordered={false} style={{ width: '100%', maxWidth: 400 }}>
          <Title level={3}>二段階認証</Title>

          {/* 情報表示 */}
          <Alert
            type="info"
            showIcon
            message="PINコードを送信しました"
            description={`メールアドレス ${email} に6桁のPINコードを送信しました。メールをご確認く
ださい。`}
            style={{ marginBottom: 24 }}
          />

          {/* エラーメッセージ */}
          {error && (
            <Alert
              type="error"
              showIcon
              closable
              message={error}
              onClose={() => setError(null)}
              style={{ marginBottom: 16 }}
            />
          )}

          <Form form={form} layout="vertical" onFinish={handleVerifyPin}>
            {/* PINコード入力 */}
```

```
          <Form.Item
            label="PINコード"
            name="pinCode"
            rules={[
              { required: true, message: 'PINコードを入力してください' },
              { pattern: /^[0-9]{6}$/, message: 'PINコードは6桁の数字です' }
            ]}
          >
            <Input.OTP
              length={6}
              size="large"
              autoFocus
              onChange={(value) => {
                // 6桁入力完了時に自動送信
                if (value.length === 6) {
                  form.submit();
                }
              }}
            />
          </Form.Item>

          {/* 有効期限タイマー */}
          <div style={{ textAlign: 'center', marginBottom: 24 }}>
            <Text type="secondary">有効期限: </Text>
            <Countdown
              value={expiresAt}
              format="mm:ss"
              valueStyle={{ fontSize: 16 }}
              onFinish={() => {
                setError('PINコードの有効期限が切れました。再送信してください');
              }}
            />
          </div>

          {/* 確認ボタン */}
          <Form.Item style={{ marginBottom: 16 }}>
            <Button
              type="primary"
              size="large"
              htmlType="submit"
              loading={isLoading}
              disabled={isLoading}
              icon={<CheckOutlined />}
              block
            >
              {isLoading ? '検証中...' : '確認'}
            </Button>
          </Form.Item>
        </Form>
```

```
          {/* 再送信リンク */}
          <div style={{ textAlign: 'center', marginBottom: 16 }}>
            <Text type="secondary">PINコードが届かない場合</Text>
            <br />
            <Button
              type="link"
              onClick={handleResendPin}
              loading={isResending}
              disabled={resendCooldown > 0 || isResending}
            >
              {resendCooldown > 0
                ? `再送信する（${resendCooldown}秒後に再試行可能）`
                : '再送信する'}
            </Button>
          </div>

          {/* 戻るボタン */}
          <Button
            type="text"
            icon={<ArrowLeftOutlined />}
            onClick={handleBack}
            block
          >
            ログイン画面に戻る
          </Button>
        </Card>
      </Space>
    </div>
  );
}
```

---

13. テスト要件

13.1. 単体テスト（Unit Test）

テスト対象

- バリデーション関数
- エラーメッセージ表示ロジック
- タイマー機能

テストケース例

```
describe('LoginForm Validation', () => {
  test('ユーザー名が空の場合、エラーメッセージを表示', () => {
    // テストコード
  });

  test('パスワードが8文字未満の場合、エラーメッセージを表示', () => {
    // テストコード
  });
});
```

13.2. 統合テスト（Integration Test）

テストシナリオ

1. ログイン成功フロー
2. パスワード認証エラー
3. PIN検証エラー
4. PIN再送信
5. タイムアウト処理

13.3. E2Eテスト（End-to-End Test）

テストツール

- Playwright（推奨）

テストシナリオ例

```
test('正常ログインフロー', async ({ page }) => {
  // ログイン画面へ遷移
  await page.goto('/login');

  // ユーザー名・パスワード入力
  await page.fill('input[name="username"]', 'test_user');
```

```
  await page.fill('input[name="password"]', 'TestPassword123!');

  // ログインボタンクリック
  await page.click('button[type="submit"]');

  // 二段階認証画面へ遷移を確認
  await page.waitForURL('/login/verify');

  // PINコード入力（モック）
  await page.fill('input[name="pinCode"]', '123456');

  // ロール別画面へ遷移を確認
  await page.waitForURL('/experiment-admin');
});
```

---

14. 変更履歴

バージョン	日付	変更内容	担当者
1.0.0	2025-12-29	初版作成	Development Team